

# Clase de algoritmos 20 de abril de 2026

¡Perfecto! Vamos a estructurar esto para que tu Obsidian no solo sea una base de datos, sino una herramienta de estudio visual y organizada. Usaremos **Callouts**, **Dataview-ready syntax** y una estructura jerárquica lógica.

---

## Sistema de Gestión Universitaria (POO en JS)

Este modelo representa la arquitectura de software para gestionar una universidad utilizando **Programación Orientada a Objetos**.

---

## Arquitectura de Clases

 **Concepto Principal** La Universidad funciona como un ecosistema donde las clases interactúan entre sí. El Estudiante se inscribe en Materias, las cuales pertenecen a una Carrera, bajo la supervisión de una Facultad.

### 1. Facultad & Carrera

La base administrativa de la institución.

JavaScript

```
class Facultad {
  constructor(nombre, decano) {
    this.nombre = nombre;
    this.decano = decano;
    this.carreras = []; // Relación 1:N
  }
}

class Carrera {
  constructor(nombre, creditosTotales, semestres) {
    this.nombre = nombre;
    this.creditosTotales = creditosTotales;
    this.semestres = semestres;
  }
}
```

### 2. Personas (Base de Datos)

Aquí aplicamos **Abstracción** (clase base para Profesor y Estudiante).

| Clase      | Atributos Clave         | Métodos / Acciones                   |
|------------|-------------------------|--------------------------------------|
| Estudiante | Correo, Celular, activo | registrarMateria(), pagarMatricula() |
| Profesor   | Especialidad, Horario   | calificar(), asignarMateria()        |

## Lógica de Negocio (Registro y Matrícula)

**📌 Registro de Materias No es solo una lista; es un objeto que vincula al Estudiante con el Profesor y la Nota.**

JavaScript

```
class RegistroMateria {
  constructor(estudiante, materia, profesor) {
    this.estudiante = estudiante;
    this.materia = materia;
    this.profesor = profesor;
    this.fechaRegistro = new Date().toLocaleDateString();
    this.notaFinal = 0;
  }

  asignarNota(valor) {
    this.notaFinal = valor;
    console.log(`Nota cargada: ${this.notaFinal} para
    ${this.estudiante.nombre}`);
  }
}
```

## Diagrama de Relaciones

Para entender cómo fluye la información en tu código:

## Ejemplo de Implementación Real

Aquí tienes cómo "instanciar" (crear) los objetos que definimos:

JavaScript

```
// 1. Definir la infraestructura
const ingenieria = new Facultad("Ingeniería", "Dr. Turing");
const ingSistemas = new Carrera("Ingeniería de Sistemas", 160, 10);

// 2. Crear los actores
const alumno1 = {
  nombre: "Juan Perez",
  correo: "juan@uni.edu",
  activo: true
};

// 3. Proceso de Matrícula
class Matricula {
  constructor(estudiante, carrera) {
    this.idMatricula = Math.random().toString(36).substr(2, 9);
    this.estudiante = estudiante;
    this.carrera = carrera;
    this.pagado = false;
  }

  confirmarPago() {
    this.pagado = true;
    console.log(`✅ Matrícula ${this.idMatricula} completada.`);
  }
}

const miMatricula = new Matricula(alumno1, ingSistemas);
miMatricula.confirmarPago();
```

---